Catalog Integration

> Common foundation for all teams. Read `README.md` first for the hackathon context.

1. Your mission

Design a solution that helps the Nordika Studio team integrate their catalog on 6 marketplaces with minimal manual effort. Your solution must handle messy data, translate content, map categories to each marketplace's taxonomy, validate the output, and give the seller clear control over what gets published.

The form of the solution is up to you — a web app, a conversational agent, a command-line tool with a rich feedback loop, a custom-trained model, a workflow of specialized agents, or anything in between. What matters is that a non-technical seller can understand, trust, and control what the system does.

What the agent must do

- Read the Nordika source catalog (CSV).
- Normalize inconsistent values if needed (casing, typos, boolean variants, separators), if necessary
- Recover missing attributes by extracting information from titles, descriptions and specifications, if necessary
- Translate titles, descriptions and localized attribute values into IT and DE.
- Map each Nordika category to an Amazon `product_type` (same for all countries) and a Google Product Category (specific to each country).
- Validate each product against the target schema (required fields, constraints, enums).
- Produce 6 compliant marketplace feeds.

What the seller must be able to do

A black-box pipeline that silently produces 6 files is not enough. Whatever form your solution takes, the seller must be able to trigger, monitor, inspect, override, and learn from the integration process. Think about: transparency of decisions, handling of edge cases, bulk vs per-SKU control, human-in-the-loop for ambiguous cases, progress feedback, error recovery.

Seller pain points to solve

- Every new product has to be integrated on 6 marketplaces, each with its own rules.
- Titles and descriptions must be translated into Italian and German.
- The internal catalog is noisy: missing values, typos, inconsistent casing.
- Marketplace validation errors mean back-and-forth edits that cost hours per SKU.

Target marketplaces

Marketplace	Country	Language	Format
Amazon FR	FR	fr-FR	JSONL
Amazon IT	IT	it-IT	JSONL
Amazon DE	DE	de-DE	JSONL

Marketplace	Country	Language	Format
Google Shopping FR	FR	fr-FR	TSV
Google Shopping IT	IT	it-IT	TSV
Google Shopping DE	DE	de-DE	TSV

---

2. Files provided

File	Description
`data/supplier_catalog_nordika_200.csv`	200-product catalog from Nordika. 80 columns. Bilingual (EN/FR). Intentionally noisy — this is part of the challenge.
`schemas/amazon_product_type.csv`	A selection of Amazon `product_type` values (~250 entries, including the 52 detailed in `amazon_product_type_attributes.yaml`).
`schemas/amazon_common_attributes.yaml`	Attributes that apply to every Amazon `product_type` (title, brand, price, dimensions, etc.). Defines types, constraints, and which attributes must be localized.
`schemas/amazon_product_type_attributes.yaml`	For each `product_type`: a description and its `specific_attributes`. Contains 52 product_types. Your job is to pick the right ones. Most specific_attributes are `recommended` (they drive ranking and filter facets); only a handful are `required` at the product_type level (e.g. `TABLE.shape`).
`schemas/google_shopping_attributes.yaml`	Google Shopping feed attributes — types, constraints, requirement levels, localization flags, and TSV encoding rules.
`schemas/google_taxonomy_fr.txt`, `schemas/google_taxonomy_it.txt`, `schemas/google_taxonomy_de.txt`	Google Shopping product taxonomies, hierarchical, one per language (public Google reference files).

Seller source catalog — key columns

- **Identifiers**: `product_id`, `brand`
- **Titles & descriptions**: `title_en`, `title_fr`, `description_en`, `description_fr`
- **Category**: `category_en`, `category_fr`
- **Colors**: `main_color`, `legs_color`, `commercial_color`, `seat_fabric_color`, `backrest_fabric_color`
- **Dimensions (numeric)**: `height_cm`, `width_cm`, `depth_cm`, `diameter_cm`, `seat_height_cm`
- **Dimensions (text)**: `dimensions_cm`, `mattress_dimensions`, `extensions_dimensions_cm`
- **Materials**: `main_material`, `legs_material`, `top_material`, `materials_detail`

- **Counts**: `nb_drawers`, `nb_doors`, `nb_shelves`, `nb_extensions`, `nb_seats`, `nb_slats`
- **Booleans**: fields prefixed with `has_` (`has_drawers`, `has_doors`, `has_storage`...)
- **Price**: `price_eur` (final selling price in EUR, what the customer sees)
- **Stock**: `stock_quantity`
- **Other**: `style`, `shape`, `function`, `assembly_required`, `room_type`, `weight_kg`, `max_load_kg`

---

3. Output format — Amazon

- **Format**: JSONL (one product per line, UTF-8)
- **Filename pattern**: `amazon_{country}_feed.jsonl` (e.g., `amazon_fr_feed.jsonl`)

> The machine-readable schema is split across **`schemas/amazon_common_attributes.yaml`** (attributes shared by all product_types) and **`schemas/amazon_product_type_attributes.yaml`** (specific_attributes per product_type) — use them if your agent needs to parse or validate attribute rules programmatically. The sections below summarize the key information for quick reading.

Locale handling

Each feed targets a single marketplace. The `marketplace_id` field on every line determines the locale for all attributes flagged `localized: true` in the YAML references. Mapping in `schemas/amazon_common_attributes.yaml > feed_metadata > marketplace_id > values`.

Country	marketplace_id
FR	`A13V1IB3VIYZZH`
IT	`APJ6JRA9NG5V4`
DE	`A1PA6795UKMFR9`

Attribute resolution

For each SKU:

1. Take all `common_attributes` with their default requirement level.
2. Add all `specific_attributes` of the chosen `product_type`.

A feed line is **valid** if every `required` attribute (common + specific) is present and satisfies its constraints.

Salient constraints

- `item_name`: 1 to 200 characters, no ALL CAPS, no promotional characters (`!`, `★`).
- `bullet_points`: 3 to 5 bullets, each 1 to 250 characters, sentence case.
- `product_description`: 1 to 2000 characters, HTML forbidden.
- Localized attributes: `item_name`, `product_description`, `bullet_points`, `color`, `material`.

Example — SKU NKS-00042

Source data (extract):

```
product_id:          NKS-00042
brand:               Nordika Studio
title_en:            Dining chair in cream boucle fabric with natural oak
legs
title_fr:            Chaise en tissu bouclette crème et pieds chêne naturel
category_en:         Chair
category_fr:         Chaise
main_color:          (empty — must be recovered from title)
legs_color:          Oak
dimensions_cm:       52 x 55 x 82
height_cm:           82.0
width_cm:            52.0
depth_cm:            55.0
seat_height_cm:      46.0
main_material:       Boucle fabric
legs_material:       Natural oak
max_load_kg:         120
weight_kg:           7.0
assembly_required:   Yes, partially assembled
price_eur:           302.0
stock_quantity:      93
```

Expected Amazon FR output (1 JSONL line, showing the common attributes only):


```
{"sku":"NKS-00042","marketplace_id":"A13V1IB3VIYZZH","product_type":"<your
choice>","brand":"Nordika Studio","item_name":"Chaise de salle à manger en
tissu bouclette crème pieds chêne naturel","product_description":"Chaise de
salle à manger contemporaine tapissée en tissu bouclette crème
doux...","bullet_points":["Tissu bouclette crème 100% polyester doux au
toucher","Pieds fuselés en chêne naturel massif","Hauteur d'assise 46 cm
adaptée aux tables standard","Charge maximale supportée : 120 kg","Livrée
partiellement assemblée"],"manufacturer":"Nordika
Studio","part_number":"NKS-00042","color":"Crème","material":"Tissu
bouclette, Chêne","item_dimensions":"52 x 55 x 82 cm","item_weight":
{"value":7.0,"unit":"kilograms"},"number_of_items":1,"condition":"new_new",
"standard_price":
{"value":302.00,"currency":"EUR"},"fulfillment_availability":
[{"fulfillment_channel_code":"DEFAULT","quantity":93}]}
```


> Most `specific_attributes` are `recommended`, not `required` — they drive ranking and filter facets rather than feed acceptance. The one exception is `TABLE.shape`, which is required because it is a primary category-browse facet. Completeness still matters (cf. §5.3), so fill every `specific_attribute` you can derive from the source: `seat_height` and `max_weight_recommendation` for CHAIR, `mattress_size` and `firmness` for MATTRESS, `number_of_drawers` for DRESSER, etc.

Expected Amazon DE title: `Esszimmerstuhl aus cremefarbenem Bouclé-Stoff mit Beinen aus natürlicher Eiche`

Expected Amazon IT title: `Sedia da pranzo in tessuto bouclé crema con gambe in rovere naturale`

---

4. Output format — Google Shopping

- **Format**: TSV (tab-separated values, UTF-8)
- **Filename pattern**: `google_{country}_feed.tsv`

> The machine-readable schema is in **`schemas/google_shopping_attributes.yaml`** — use it if your agent needs to parse or validate attribute rules programmatically. The tables below summarize the same information for quick reading.

Locale handling

Each feed targets a single locale declared in `feed_metadata.feed_locale` (`fr_FR`, `it_IT`, `de_DE`). All attributes flagged `localized: true` in the YAML must be expressed in that language — including `title`, `description`, `color`, `material`, `product_type`, and the localized form of `google_product_category`.

Required columns

Column	Description
`id`	Unique SKU, e.g., `NKS-00042`
`title`	1 to 150 chars, in target language
`description`	1 to 5000 chars, in target language
`availability`	Enum: `in_stock`, `out_of_stock`, `preorder`, `backorder`
`price`	Format: `302.00 EUR` — amount space currency
`brand`	`Nordika Studio`
`condition`	Enum: `new`, `refurbished`, `used`
`google_product_category`	Google taxonomy node (numeric ID or full localized path)

Recommended columns

Column	Description
`gtin`	If available
`mpn`	Manufacturer part number (usually the SKU)
`color`	Localized color label
`material`	Localized material label
`product_type`	Free categorical path, e.g., `Meubles > Salon > Canapés`
`shipping`	Format: `FR::Standard:9.90 EUR`

Column	Description
`shipping_weight`	Format: `7.0 kg`

Constraints

- `title` and `description` must be in the target-country language.
- `color` and `material` must be localized (e.g., `Chêne` for FR, `Rovere` for IT, `Eiche` for DE).
- `price` must use the format `<amount> EUR` (e.g., `302.00 EUR`), same price across all locales.
- TSV encoding: no raw tabs inside values, line breaks escaped as literal `\n`, multi-value fields (e.g. `product_highlight`) use `|` as separator.

Example — SKU NKS-00042, Google FR row

```
NKS-00042   Chaise de salle à manger en tissu bouclette crème pieds chêne
Chaise de salle à manger contemporaine tapissée en tissu bouclette crème...
in_stock    302.00 EUR  <your choice>   Nordika Studio  NKS-00042   new
Crème   Tissu bouclette, Chêne  <your choice>   FR::Standard:9.90 EUR   7.0
kg
```

---

5. Business rules

Stock

Each product has an initial stock quantity in the `stock_quantity` column of the source catalog. Use it as-is for the initial integration.

- For Amazon, populate `fulfillment_availability[0].quantity` with `stock_quantity`.
- For Google, derive `availability`: `in_stock` if `stock_quantity > 0`, otherwise `out_of_stock`.
- A small number of SKUs will have `stock_quantity = 0` — your agent must decide whether to publish them anyway (as `out_of_stock`) or skip them entirely. Both are defensible; document the choice.

Stock management across marketplaces (allocation, restocking, sync) is the subject of the *Order processing & supply* deep dive.

Pricing

Each product has a single price in the `price_eur` column — this is the final selling price in EUR, the same on all marketplaces, for all countries. Publish it as-is. No margin computation, no VAT handling, no currency conversion to manage here.

Price optimization is the subject of the *Dynamic pricing* deep dive.

Translation

- **For FR markets**: use `title_fr` and `description_fr` as-is.

- **For IT and DE markets**: translate from `title_en` and `description_en`. English is the safer pivot because the French text sometimes contains encoding artifacts and typos.
- **For localized attribute values**: an attribute flagged `localized: true` must also be translated (e.g., color `Cream` becomes `Crème` in FR, `Crema` in IT, `Creme` in DE).

Required is a minimum, not a target ⚠️

Passing validation with only the required fields filled is **not a success**. Marketplaces rank and filter products based on how complete their listings are:

- On Amazon, listings with rich attributes (`material`, `color`, `style`, `max_weight_recommendation`, etc.) appear in more filter facets and rank higher in relevance. A product with only the bare minimum is often invisible in category browse pages.
- On Google Shopping, the recommended columns (`color`, `material`) directly affect ad eligibility and CTR. Products missing them show up less in shopping ads.
- Bullet points and detailed descriptions drive organic conversions. "Valid but empty" listings convert poorly.

Rule of thumb for your agent: fill every attribute you can reasonably derive from the source data — from the structured columns, from free text extraction, or from sensible defaults. Required fields are the floor; completeness is what sells.

---

6. Deliverables

Feed files — proof that the integration works

Your integration is considered successful when your solution can produce these 6 files. They are the tangible output of your pipeline, the thing a reviewer (or Nordika's founder) can open and inspect to verify that the plumbing is real.

File	Content
`amazon_fr_feed.jsonl`	Amazon FR feed, one product per line
`amazon_it_feed.jsonl`	Amazon IT feed
`amazon_de_feed.jsonl`	Amazon DE feed
`google_fr_feed.tsv`	Google Shopping FR feed
`google_it_feed.tsv`	Google Shopping IT feed
`google_de_feed.tsv`	Google Shopping DE feed

Treat these files as **a checkpoint, not a final deliverable**. Getting them right means your mapping, translation, normalization and validation logic all work end-to-end. Getting them wrong (missing fields, wrong language, broken enums) tells you exactly where your pipeline breaks.

> **Tip:** writing a small auto-check that parses your feeds and verifies them against the YAML schemas (required fields present, enums valid, lengths within bounds, locale correct) is a cheap way to catch

> regressions while you iterate. Bonus if the seller can trigger it from your solution and get a pass/fail report per marketplace.

---

7. Good to know

- The full attribute requirements depend on the chosen `product_type`. Picking `FURNITURE` as a catch-all is technically valid but makes your listings poor. Picking a too-specific type may force requirements you cannot satisfy.
- Bullet points don't exist in the source catalog. You must generate them from `description_*` and `specifications`.
- Bonus — watch out for scale: a naive "one LLM call per SKU per marketplace" means 1,200 calls just for 200 products. Think about caching, batching, and which parts actually need an LLM vs simple rules.